

Resenha Crítica: O Mapa da Arquitetura e o Desafio da Integração

Autor da Resenha: Bernardo Gonçalves Levy

Disciplina: Projeto de Software

Artigo Analisado: *Software Architecture: a Roadmap* (David Garlan, 2000)

1. Visão Geral: O Passado, o Presente e o Futuro da Arquitetura

No artigo "*Software Architecture: a Roadmap*", publicado no ano 2000, David Garlan faz um mapeamento abrangente do campo da arquitetura de software, traçando sua evolução de uma prática *ad hoc* para uma disciplina formal de engenharia. O autor destaca como a indústria passou a reconhecer que o design de alto nível de um sistema — suas estruturas, componentes e interações — é o principal fator que determina atributos de qualidade como desempenho, confiabilidade e capacidade de evolução.

Garlan divide sua análise entre o estado da arte da época e as direções futuras. No presente de 2000, ele aponta sucessos como a catalogação de estilos arquiteturais (como *Pipes and Filters* e arquiteturas em camadas), o desenvolvimento de Linguagens de Descrição de Arquitetura (ADLs) e o surgimento de linhas de produtos de software. Já ao olhar para o futuro, o autor prevê de forma quase profética os desafios da integração de sistemas heterogêneos, a necessidade de arquiteturas dinâmicas que se adaptam em tempo de execução e a ascensão dos ecossistemas baseados em componentes, onde o software é cada vez mais "montado" a partir de partes pré-existentes em vez de ser construído do zero.

2. Análise Crítica: O Paradigma da Incompatibilidade de Componentes

Lendo o "mapa" de Garlan hoje, é impressionante constatar a precisão de suas previsões, especialmente no que tange ao desenvolvimento baseado em componentes (*Component-Based Software Engineering* - CBSE). Um dos pontos mais críticos levantados pelo autor é o problema do *component mismatch* (incompatibilidade arquitetural). Garlan alerta que o grande desafio do futuro não seria escrever novos algoritmos, mas sim fazer com que peças de software construídas por equipes diferentes, com premissas diferentes, funcionassem juntas de forma harmoniosa.

A teoria acadêmica anterior muitas vezes assumia que os desenvolvedores tinham controle total sobre o código-fonte de todo o sistema. Garlan desconstrói essa ilusão, mostrando que

a arquitetura moderna é, na verdade, um exercício de negociação e restrição. Quando utilizamos bibliotecas, plataformas ou serviços de terceiros, herdamos também suas decisões arquiteturais embutidas. Se essas premissas entram em conflito com os requisitos do nosso sistema, o papel do arquiteto deixa de ser o de um "criador" e passa a ser o de um "integrador", desenvolvendo pontes, adaptadores e fachadas para mitigar essas incompatibilidades.

3. Aplicação Prática: O "Component Mismatch" na Offerwise

Trazendo as reflexões de Garlan para o ambiente da programação de pesquisas de mercado na Offerwise, o conceito de *component mismatch* se materializa diariamente. O cenário de desenvolvimento de questionários é o exemplo perfeito de um ecossistema de componentes heterogêneos.

A plataforma Qualtrics atua como o grande componente central, possuindo uma arquitetura robusta, otimizada para estabilidade e escala, mas que possui regras de negócio genéricas. O desafio surge quando as instruções da pesquisa exigem uma lógica que a plataforma não prevê nativamente — como uma validação de cota cruzada em tempo real ou uma exclusão demográfica altamente específica. Ocorre aqui a incompatibilidade arquitetural descrita por Garlan: o componente de prateleira (Qualtrics) não atende aos requisitos exatos do sistema final.

Para resolver esse *mismatch*, o trabalho com jQuery atua como a "cola arquitetural". O desenvolvimento não acontece a partir do zero; ele ocorre nas bordas do sistema:

- **Arquiteturas Adaptativas:** Garlan previu sistemas que precisariam se adaptar dinamicamente. Os *scripts* injetados via JavaScript fazem exatamente isso, manipulando o DOM (*Document Object Model*) em tempo de execução para esconder opções, pular perguntas ou alertar o usuário, adaptando a interface rígida às necessidades fluidas da pesquisa.
- **O Risco da Integração Forçada:** Como Garlan adverte, integrar componentes com premissas diferentes gera atrito. Ao utilizar jQuery para modificar o comportamento do Qualtrics, cria-se uma dependência não oficial da estrutura HTML da plataforma. Isso exige uma arquitetura de *front-end* defensiva, garantindo que o *script* funcione sem quebrar o fluxo central de envio de dados do questionário.

4. Conclusão

O artigo "*Software Architecture: a Roadmap*" demonstra que a maturidade na engenharia de software reside em compreender e gerenciar sistemas compostos. A arquitetura moderna, como Garlan brilhantemente antecipou, lida com a realidade de que raramente temos controle sobre todas as engrenagens do motor.

No contexto ágil da coleta de dados corporativos, o sucesso não depende de reinventar a roda, mas de dominar a arte da integração. Compreender as limitações e as premissas arquiteturais das plataformas que utilizamos — e saber construir *scripts* resilientes para

suprir essas lacunas — é o que garante que as pesquisas cheguem a campo com estabilidade, provando que o verdadeiro design de software hoje é, acima de tudo, a ciência da adaptação.